JavaScript Theory Question

Core JavaScript Fundamentals

Q1. What is JavaScript and why use it

JavaScript is a high-level, interpreted, dynamically typed programming language primarily used to create interactive and dynamic web applications. It is one of the core web technologies alongside HTML and CSS, and it runs natively in all modern web browsers like Google Chrome.

JavaScript follows a prototype-based object model and supports multiple programming paradigms, including event-driven, functional, and object-oriented programming.

Why is JavaScript used?

1. Client-side interactivity
JavaScript enables dynamic behavior in web pages, such as DOM manipulation, event handling, form validation, and animations—improving user experience.

2. Asynchronous programming
It supports asynchronous operations using callbacks, promises, and async/await, allowing non-blocking execution (e.g., API calls without freezing the UI).

3. Cross-platform compatibility
JavaScript runs in all major browsers and environments, making it universally accessible.

4. Full-stack development
With environments like Node.js, JavaScript can be used on the server side, enabling developers to use a single language for both frontend and backend.

5. Rich ecosystem
It has a vast ecosystem of libraries and frameworks like React, Angular, and Vue, which accelerate development.

Q2. Explain how many types of javascript?

1. Client-Side JavaScript

- Runs in the user's browser (like Google Chrome)
- Used to create interactive UI elements
- Works with the DOM (Document Object Model)

Examples:

- Form validation
- Button click events
- Animations and UI updates

---

2. Server-Side JavaScript

- Runs on the server using environments like Node.js
- Used for backend logic, APIs, and database handling

Examples:

- Handling HTTP requests
- Connecting to databases
- Authentication systems

Q3. How can we run javascript in a server side environment ?
Steps to run on server side environment

1. Install Node.js
2. Create a JavaScript file (e.g., app.js)
3. Write server-side code
4. Run using command

Q4. What is dynamic type and static type language?

◆ **Static Typed Language**

A **statically typed language** is one where the **data type of a variable is declared at compile time**, and it cannot change later.

Key points:

- Type is checked before execution (compile time)
- Errors are caught early
- Improves performance and code reliability

Example languages:

- Java
- C++
- TypeScript (a superset of JavaScript) → TypeScript

Example:
int num = 10; // must always be an integer

◆ **Dynamic Typed Language**

A **dynamically typed language** is one where the **data type is determined at runtime**, and variables can hold different types at different times.

Key points:

- No need to declare type explicitly
- Type checking happens during execution
- More flexible but can cause runtime errors

Example languages:

- JavaScript
- Python

Example:
```
let x = 10;     // number
x = "Hello";    // now string
```

Q5. What are the different data types in JavaScript?

Q6. Difference between var, let, and const?

Q7.What is hoisting?
Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their scope during the compilation phase, before code execution. This means you can use variables and functions before they are actually declared in the code.

How Hoisting Works

1. **Function declarations** are fully hoisted, so you can call the function before it appears in the code.
2. **Variables declared with var** are hoisted but initialized as undefined.
3. **Variables declared with let and const** are hoisted but **not initialized**, resulting in a **Temporal Dead Zone (TDZ)** until their declaration is encountered.

Example1:
```
console.log(a); // undefined
var a = 10;
```

Example2:
```
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;
```

Q8.What is scope in JavaScript?

Scope determines where a variable or function is accessible in your code. It defines the visibility and lifetime of variables.

Types of Scope in JavaScript

1. Global Scope

    - Variables declared outside any function are in the global scope.
    - Accessible from anywhere in the code.

Example:
```
var globalVar = "I am global";
console.log(globalVar); // "I am global"
```

Function (Local) Scope

- Variables declared inside a function are accessible only within that function.

Example:
```
function myFunc() {
  var localVar = "I am local";
  console.log(localVar); // "I am local"
}
console.log(localVar); // ReferenceError
```

Block Scope

- Introduced with let and const.
- Variables are accessible **only within the block {}** they are defined in.

Example:

```
{
  let blockVar = "I am block scoped";
  const blockConst = 100;
  console.log(blockVar); // works
}
console.log(blockVar); // ReferenceError
```

Q9. Difference between global scope and block scope?

Difference Between Global Scope and Block Scope in JavaScript

Feature	**Global Scope**	**Block Scope**
Definition	Variables declared outside any function or block. Accessible anywhere in the code.	Variables declared inside {} (like in loops, if statements, or blocks) using let or const. Accessible only within that block.

Keyword Support	var (can also be global), let, const	let, const (not var for true block scope)
Accessibility	Accessible from any function or block in the code	Accessible only within the block they are defined
Lifetime	Exists for the lifetime of the program	Exists only during the execution of the block

Q10. What are closures?

A closure is a function that remembers and has access to variables from its outer (enclosing) scope, even after the outer function has finished executing. In simpler terms: a function "closes over" the variables of its parent scope.

How Closures Work

1. JavaScript functions form a lexical scope (scope defined where they are written).
2. If an inner function accesses variables from the outer function, those variables stay alive as long as the inner function exists.

```
function outerFunction() {
  let count = 0; // variable in outer scope

  return function innerFunction() {
    count++;
    console.log(count);
  }
}

const counter = outerFunction();
counter(); // 1
counter(); // 2
counter(); // 3
```

Q11. What is a callback function?

A callback function is a function that is passed as an argument to another function and is executed after some operation has completed.

It allows you to control the order of execution in asynchronous or event-driven code.

How Callbacks Work

- You define a function
- Pass it as an argument to another function
- That function calls (executes) it at the appropriate time

Example

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

function sayGoodbye() {
  console.log("Goodbye!");
}

greet("Sam", sayGoodbye);
```

Important Points for Interviews

- Callbacks help manage asynchronous operations (like API calls or timers)
- Avoid blocking the main thread
- Can lead to “callback hell”, which can be solved using Promises or async/await

Q12. What is a higher-order function?

A higher-order function (HOF) is a function that can take other functions as arguments, return a function, or both. In simple terms, functions that operate on functions are higher-order functions.

Why Higher-Order Functions Are Useful

- Enable code reusability
- Simplify asynchronous programming
- Core concept in functional programming

Example

```
function greet(name, formatter) {
  console.log("Hello " + formatter(name));
}

function upperCaseName(name) {
  return name.toUpperCase();
}
greet("Sam", upperCaseName); // Output: Hello SAM
```

Q13.Difference between == and ===?

1. Type Checking
    - == → Compares values only, ignores type differences (type coercion happens)
    - === → Compares both value and type (no type coercion)
2. Type Conversion

    - == → Performs implicit type conversion if types differ
    - === → No type conversion; must match type exactly
3. Safer to Use
    - == → Less safe, can lead to unexpected results
    - === → Safer, recommended in modern JavaScript
4. Examples
    - 5 == '5' → true
    - 5 === '5' → false
    - null == undefined → true
    - null === undefined → false

Q14. What are truthy and falsy values?

In JavaScript, **all values are either truthy or falsy** when evaluated in a **boolean context** (like if, while, !, etc.).

- **Truthy** → Values that evaluate to true
- **Falsy** → Values that evaluate to false

Falsy Values

There are **exactly 7 falsy values** in JavaScript:

Value	Description
false	Boolean false
0	Number zero
-0	Negative zero
"" or ''	Empty string
null	Null value
undefined	Undefined value

Truthy Values

- All values that are **not falsy** are truthy
- Examples:
    - Non-empty strings: "hello"

- Non-zero numbers: 1, -1
- Arrays: []
- Objects: {}
- Functions: function(){}

Q15. What is NaN?

NaN stands for “Not-a-Number”. It is a special value in JavaScript that represents a value that is not a legal number.

important Points

1. **Type**
   - NaN is of type **number**:
2. console.log(typeof NaN); // "number"
3. **When does NaN occur?**
   - Performing **invalid arithmetic operations**:
4. let result = "hello" * 2; // NaN
   - Trying to convert **non-numeric strings** to numbers:
5. Number("abc"); // NaN
6. **Special Characteristics**
   - NaN **is not equal to anything**, including itself:
7. console.log(NaN === NaN); // false
   - To check for NaN, use:
     - isNaN(value) → checks if value **cannot be converted to a number**
     - Number.isNaN(value) → stricter, checks if value **is actually NaN**

◆ **Example**

```
console.log(0 / 0);          // NaN
console.log(Math.sqrt(-1));    // NaN
console.log(Number("abc"));    // NaN
```

Q17.What is undefined vs null?

Difference Between undefined and null

1. **Definition**
   - **undefined → A variable declared but not assigned any value.**
   - **null → A variable explicitly assigned “no value” by the programmer.**
2. **Type**
   - **undefined → typeof gives "undefined"**
   - **null → typeof gives "object"**
3. **Assigned By**
   - **undefined → Automatically by JavaScript**
   - **null → Manually by the developer**
4. **Purpose / Usage**
   - **undefined → Indicates variable exists but value not yet assigned**
   - **null → Indicates intentional absence of value**

5. **Comparison**
    - **undefined == null → true (loose equality)**
    - **undefined === null → false (strict equality)**
6. **Examples**

```
let a;
console.log(a);      // undefined
let b = null;
console.log(b);
```

Q18.What is the typeof operator?

The typeof operator in JavaScript is used to determine the data type of a variable or value. It returns a string indicating the type.
Syntax: typeof operand
Example:

```
console.log(typeof 10);          // "number"
console.log(typeof "Hello");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object" (this is a historical quirk)
console.log(typeof {});           // "object"
console.log(typeof function(){});  // "function"
```

important Points for Interviews

1. Returns a string representing the type of the operand.
2. typeof null returns "object" → historical bug, be careful.
3. Can be used to check primitive types (number, string, boolean, undefined, symbol, bigint) and objects.
4. Safe to use on undeclared variables in modern JavaScript:

Q19.What is an IIFE (Immediately Invoked Function Expression)?

An IIFE (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined.

- It is wrapped in parentheses to make it a function expression
- Immediately followed by () to invoke it

Syntax

```
(function() {
  console.log("This runs immediately!");
})();
```

Or using arrow function:

```
(() => {
  console.log("Arrow IIFE runs immediately!");
})();
```

Why Use IIFE?

1. **Avoids global scope pollution**
    - Variables inside the IIFE are **private to the function**
2. **Encapsulation**
    - Helps create a **local scope** for code that executes immediately
3. **Initialization code**
    - Useful for code that only needs to run **once**

Example with source code

```
const result = (function(x, y) {
  return x + y;
})(5, 10);

console.log(result); // 15
```

Q20. What is the spread operator?

The **spread operator (...)** allows an **iterable (like an array or string) to be expanded** into individual elements.

It is used to **copy, merge, or pass elements** without manually looping.
Syntax:

1. Expanding an array

```
const numbers = [1, 2, 3];
console.log(...numbers); // 1 2 3
```

2. Copying an array

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1]; // [1, 2, 3]
```

3. Merging arrays

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2]; // [1, 2, 3, 4]
```

4. Copying objects

```
const obj1 = {a: 1, b: 2};
const obj2 = {...obj1, c: 3}; // {a:1, b:2, c:3}
```

5. Function arguments

```
function sum(x, y, z) {
  return x + y + z;
}

const numbers = [1, 2, 3];
console.log(sum(...numbers)); // 6
```

Q21. What is the rest parameter?

The rest parameter (...) allows a function to accept an indefinite number of arguments as an array.

- **It is used to collect all remaining arguments into a single variable**
- **Only one rest parameter is allowed, and it must be the last parameter in the function**

Example with source code

```
function myFunc(a, b, ...rest) {
  console.log(a);     // first argument
  console.log(b);     // second argument
  console.log(rest);  // array of remaining arguments
}
```

Example

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}
console.log(sum(1, 2, 3));       // 6
console.log(sum(5, 10, 15, 20));  // 50
```

Q22. What are template literals?

Template literals are a way to create strings in JavaScript using backticks (` `) instead of quotes (' ' or " ").

- They allow embedded expressions, multi-line strings, and string interpolation.
- Introduced in ES6 (ECMAScript 2015).

Example with source code

```
`string text ${expression} string text`
```

Features

1. **String Interpolation**

```
const name = "Sam";
const greeting = `Hello, ${name}!`;
console.log(greeting); // "Hello, Sam!"
```

2. **Multi-line Strings**

```
const text = `This is line 1
This is line 2`;
console.log(text);
```

3. **Expression Evaluation**

```
const a = 5, b = 10;
console.log(`Sum is ${a + b}`); // "Sum is 15"
```

4. Tagged Templates (Advanced)

```
function tag(strings, value) {
 return `${strings[0]}${value * 2}`;
}
console.log(tag`Double of 5 is ${5}`); // "Double of 5 is 10"
```

Q23.What is destructuring?

Destructuring is a JavaScript feature that allows you to extract values from arrays or properties from objects into distinct variables in a concise way.

- Introduced in ES6 (ECMAScript 2015)
- Makes code cleaner, shorter, and easier to read

◆ **Array Destructuring**

```
const numbers = [1, 2, 3];
const [a, b, c] = numbers;
console.log(a); // 1
console.log(b); // 2
console.log(c); // 3

const person = { name: "Sam", age: 25 };
const { name, age } = person;
console.log(name); // "Sam"
console.log(age);  // 25
```

Object destructing

```
const person = { name: "Sam", age: 25 };
const { name, age } = person;

console.log(name); // "Sam"
console.log(age);  // 25
```

Function parameter destructing

```
function greet({ name, age }) {
  console.log(`Hello ${name}, age ${age}`);
}
greet({ name: "Sam", age: 25 }); // "Hello Sam, age 25"
```

Q24.How does JavaScript execution context work?

An **execution context (EC)** is an abstract concept that holds information about the environment in which the current code is being executed. It determines **what variables, functions, and objects are accessible** at a given point in time.

There are **three main types**:

1. **Global Execution Context (GEC)**
   - Created when the JavaScript engine first starts executing a script.
   - There's only **one global context**.
   - It creates a global object (window in browsers, global in Node.js) and a special variable this.
   - All code outside functions runs here.
2. **Functional Execution Context (FEC)**
   - Created whenever a function is invoked.
   - Each function has its **own EC**, even if nested inside another function.
3. **Eval Execution Context**
   - Rarely used; created when code is executed inside eval().

Phases of Execution Context

Every execution context has **two main phases**:

1. Creation Phase

- **Hoisting** happens here:
  - Function declarations are fully hoisted.
  - Variable declarations are hoisted but initialized with undefined.
- **Scope chain** is created.
- this is determined.

Example

```
console.log(a); // undefined
var a = 5;
function greet() {
  console.log("Hello!");
}
greet(); // works fine
```

Execution Phase

- Code is executed line by line.
- Variables are assigned actual values.
- Functions are executed when called.

Scope Chain & Lexical Environment

- Each EC has a **Lexical Environment** containing:

    - Environment Record → stores variable & function declarations.
    - Outer Lexical Environment Reference → points to the parent scope.
- This forms the **scope chain**, which is used for identifier resolution.

Call Stack

- JavaScript uses a **single-threaded call stack**.
- When a function is called, a **new execution context** is pushed onto the stack.
- After execution, it is popped off.
- The global context stays at the bottom until the program ends.

Example

```
function first() {
  second();
  console.log("First done");
}
function second() {
  console.log("Second executed");
}
first();
```

Call stack flow:

1. Global EC created → first() called.
2. first() EC pushed → second() called.
3. second() EC pushed → executes → popped.
4. first() EC resumes → executes → popped.
5. Global EC finishes.

Q25.What is the call stack?

The call stack is a data structure (stack) used by the JavaScript engine to keep track of function execution.
Since JavaScript is single-threaded, it can only execute one function at a time, and the call stack ensures the engine knows which function is currently running and what to execute next.

Think of it like a stack of plates:

- You push a function onto the stack when it's called.
- You pop it off the stack when it finishes execution.

How It Works

1. Global Execution Context is created and pushed onto the stack.
2. When a function is called, a new execution context for that function is created and pushed on top.
3. Execution happens in LIFO order (Last In, First Out).

4. After the function finishes, its context is popped off the stack.
5. The engine resumes execution in the previous context.

Example

```
function first() {
   console.log("Inside first");
   second();
   console.log("First done");
}

function second() {
   console.log("Inside second");
}

console.log("Start");
first();
console.log("End");
```

Q26.What is recursion?

Recursion is when a function calls itself, either directly or indirectly, to solve a problem.
It is commonly used to solve problems that can be broken down into smaller, similar subproblems.

Key Parts of a Recursive Function

1. Base Case – The condition that stops the recursion, preventing infinite calls.
2. Recursive Case – The part where the function calls itself with a modified argument.

Without a base case, the function will call itself indefinitely, leading to a stack overflow.

Example

```
function factorial(n) {
   if (n === 0) { // base case
      return 1;
   }
   return n * factorial(n - 1); // recursive case
}

console.log(factorial(5)); // Output: 120
```

Q27.Difference between function declaration and function expression?

1. Function Declaration (FD)

Definition:
A function declared using the function keyword with a name.

Syntax:

```
function greet() {
  console.log("Hello!");
}
```

Key Features:

1. Hoisting:
    - Function declarations are fully hoisted, meaning they can be called before they appear in code.
2.
```
greet(); // Works!
function greet() {
   console.log("Hello!");
}
```
3. Name:
    - Must have a function name.
4. Scope:
    - Part of the enclosing scope, usually global if declared at top level.

---

2. Function Expression (FE)

Definition:
A function assigned to a variable (can be named or anonymous).

Syntax:

```
const greet = function() {
  console.log("Hello!");
};
```

Key Features:

1. Hoisting:
    - Only the variable is hoisted, not the function assignment.
    - Calling it before the line throws an error.
2.
```
greet(); // Error: Cannot access 'greet' before initialization
const greet = function() {
   console.log("Hello!");
};
```

3. Anonymous or Named:

```
const greet = function sayHello() {
  console.log("Hello!");
};
```

4. Scope:
   - Behaves like a variable; obeys block scope if let or const is used.

Q28.What are arrow functions?

Definition:
Arrow functions (=>) are a shorter syntax for writing functions introduced in ES6. They are anonymous by default and do not have their own this, arguments, super, or new.target.

Syntax:

```
// Basic syntax
const add = (a, b) => a + b;

console.log(add(2, 3)); // Output: 5
```

Key Features of Arrow Functions

1. Concise Syntax
   - If the function has a single expression, you can omit {} and return.
2. const square = x => x * x;

3. this Binding

   - Arrow functions do not have their own this; they inherit this from the lexical scope.
4.
```
const obj = {
  value: 10,
  arrowFunc: () => console.log(this.value),
  regularFunc() { console.log(this.value); }
};

obj.arrowFunc(); // undefined (inherits from global)
obj.regularFunc(); // 10
```

5. No arguments object

    - Arrow functions do not have arguments. Use rest parameters (...args) instead.

6. const sum = (...args) => args.reduce((a,b) => a+b, 0);

7. Cannot be used as constructors

    - You cannot use new with arrow functions.

8. const Foo = () => {};
   new Foo(); // Error

9. Cannot use yield
    - Arrow functions cannot be generators.

Q29.Difference between arrow functions and regular functions?

Arrow Functions vs Regular Functions

1. Syntax
    - Arrow: (a, b) => a + b (concise, shorter)
    - Regular: function(a, b) { return a + b; }
2. this Binding
    - Arrow: Lexical this → inherits this from the enclosing scope
    - Regular: Dynamic this → depends on how the function is called
3. arguments Object
    - Arrow: ❌ Not available (use rest parameters ...args instead)
    - Regular: ✅ Available by default
4. Constructor
    - Arrow: ❌ Cannot be used with new
    - Regular: ✅ Can be used with new
5. Hoisting
    - Arrow: Behaves like a variable → not hoisted (TDZ applies with let/const)
    - Regular: Fully hoisted → can be called before declaration
6. Use as Generators
    - Arrow: ❌ Cannot be a generator function
    - Regular: ✅ Can be used with function*
7. super and new.target
    - Arrow: ❌ Not available
    - Regular: ✅ Available
8. Ideal Use Case
    - Arrow: Short callbacks, array methods, preserving lexical this
    - Regular: General-purpose functions, object methods, dynamic this behavior

Q30.What is this keyword?

This keyword refers to the **context object** in which a function is executed.

- Its value **depends on how the function is called**, not where it is defined.
- It allows a function to access properties and methods of its **current execution context**.

Rules for this

1. Global Context

- Outside of any function, this refers to the **global object**:
  - Browser → window
  - Node.js → global

```
console.log(this); // window (in browser)
```

---

2. Inside a Regular Function

- **Non-strict mode:** this → global object
- **Strict mode ('use strict'):** this → undefined

```
function show() {
  console.log(this);
}
show(); // window (non-strict) / undefined (strict)
```

---

3. Inside an Object Method

- this refers to the **object that owns the method**

```
const obj = {
  name: "Alice",
  greet: function() {
    console.log(this.name);
  }
};
obj.greet(); // "Alice"
```

---

4. Inside a Constructor Function

- When using new, this refers to the **newly created object**

```
function Person(name) {
  this.name = name;
}
const p = new Person("Bob");
console.log(p.name); // "Bob"
```

---

5. Arrow Functions

- Arrow functions **do not have their own this**
- this is **lexically inherited** from the enclosing scope

```
const obj = {
  name: "Alice",
  arrowFunc: () => console.log(this.name)
};
obj.arrowFunc(); // undefined (inherits global this)
```

---

6. this in call, apply, and bind

- You can **manually set this** using:
    - func.call(obj, arg1, arg2)
    - func.apply(obj, [arg1, arg2])
    - const boundFunc = func.bind(obj)

```
function greet() { console.log(this.name); }
const person = { name: "Charlie" };

greet.call(person); // "Charlie"
```

Q31.What is function currying?

_____

Function currying is the process of transforming a function that takes multiple arguments into a sequence of functions, each taking one argument at a time.

- Instead of f(a, b, c), you get f(a)(b)(c).
- Currying allows partial application of functions, i.e., fixing some arguments in advance.

Why use Currying?

1. Reusability: Create specialized functions by pre-filling some arguments.
2. Functional Programming: Enables composition of small functions.
3. Clean Code: Avoids passing repeated arguments multiple times.

Example 1: Basic Currying

```
function add(a) {
   return function(b) {
      return a + b;
   };
}

console.log(add(2)(3)); // 5
```

Example 2: Using Arrow Functions

```
const multiply = a => b => c => a * b * c;
console.log(multiply(2)(3)(4)); // 24
```

Q32.What is a partial application?

Partial application is a technique in which a function is called with fewer arguments than it expects, producing a new function that takes the remaining arguments.

- It allows you to "preset" some arguments while keeping the function flexible for the rest.
- Unlike currying, partial application does not necessarily take one argument at a time; it just fixes some arguments.

Example with source code

```
function multiply(a, b, c) {

  return a * b * c;

}

// Partially apply a = 2

function multiplyBy2(b, c) {

  return multiply(2, b, c);

}


console.log(multiplyBy2(3, 4)); // 24
```

Q33.What are pure functions?

A pure function is a function that:

1. Always returns the same output for the same input.
2. Has no side effects (does not modify external state, global variables, or input arguments).

In short: output depends only on input, and it does not change anything outside itself.

Key Characteristics

1. Deterministic – Same input → same output.
2. No Side Effects – Does not modify:
    - Global variables
    - Parameters (immutable)
    - DOM or external state

3. Easier to test – Predictable behavior.
4. Functional Programming Friendly – Can be composed, cached, or memoized.

Example

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // 5
console.log(add(2, 3)); // 5 (always same output)
```

Q34.What is memoization?

Memoization is an optimization technique used to cache the results of expensive function calls so that future calls with the same inputs return the cached result instead of recomputing.

- Improves performance, especially for recursive or computationally heavy functions.
- Often used in dynamic programming problems.

Important Points

1. Stores **input-output mapping** in memory.
2. Reduces **time complexity** at the cost of **space (memory)**.
3. Works best for **pure functions**, because same input always produces same output.

Example with source code

```
function fibonacci(n, cache = {}) {

  if (n <= 1) return n;

  if (cache[n]) return cache[n]; // return cached value

  cache[n] = fibonacci(n - 1, cache) + fibonacci(n - 2, cache); // store result

  return cache[n];

}

console.log(fibonacci(10)); // 55
```

Note: Without memoization, recursive Fibonacci has exponential time complexity O(2^n).

With memoization, it becomes O(n).

Example 2: Generic Memoization Function

```
function memoize(fn) {   const cache = {};

  return function(...args) {
```

```
    const key = args.toString();

    if (cache[key]) {

      return cache[key];

    }

    const result = fn(...args);

    cache[key] = result;

    return result;

  };

}

const square = memoize(x => x * x);

console.log(square(4)); // 16 (computed)

console.log(square(4)); // 16 (cached)
```

Q35. What are arrays in JavaScript?

- An array is a special type of object used to store ordered collections of values.
- The values can be of any type: numbers, strings, objects, other arrays, functions, etc.
- Arrays are zero-indexed, meaning the first element is at index 0.

Key Features

1. Dynamic Size
   - Arrays can grow or shrink dynamically.
2. Heterogeneous Elements
   - Can store different types of data in the same array.
3. Indexed Access
   - Access elements via numeric indices: arr[0], arr[1].
4. Array Methods
   - JavaScript provides built-in methods like push, pop, shift, unshift, map, filter, reduce, etc.

Example

```
const fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // "Apple"
console.log(fruits.length); // 3
fruits.push("Orange"); // Add to the end
console.log(fruits); // ["Apple", "Banana", "Mango", "Orange"]
fruits.pop(); // Remove last element
console.log(fruits); // ["Apple", "Banana", "Mango"]
```

Q36.Difference between array and object?

Purpose

- Array: Stores ordered collections of values.
- Object: Stores key-value pairs for unstructured data.

Indexing

- Array: Accessed via numeric indices (0, 1, 2…).
- Object: Accessed via keys (strings or symbols).

Order

- Array: Maintains element order.
- Object: Does not guarantee order (except in modern JS engines for string keys).

Accessing Elements

- Array: arr[0], arr[1]
- Object: obj.key or obj["key"]

Iteration

- Array: for, forEach, map, filter, for…of
- Object: for…in, Object.keys(), Object.values(), Object.entries()

Length / Size

- Array: .length property gives number of elements.
- Object: Object.keys(obj).length gives number of properties.

Methods / Built-in Functions

- Array: Has built-in methods like push, pop, shift, unshift, map, filter, reduce.
- Object: No built-in array-like methods; use Object.keys/values/entries.

Use Cases

- Array: Lists, stacks, queues, ordered datasets.
- Object: Dictionaries, configurations, key-value storage.

Type

- Array: Type is "object" but Array.isArray(arr) returns true.
- Object: Type is "object"

MCQ Question on JavaScript

Part 1: Coding MCQs (Detailed Explanation)

Q1.

```
console.log(a);
var a = 10;
```

Options:
A. 10
B. undefined
C. ReferenceError
D. null

Answer: B

Explanation:

- A is incorrect because the assignment `a = 10` happens after the log statement during execution.
- B is correct because `var` variables are hoisted and initialized with `undefined` in the creation phase.
- C is incorrect since `var` does not throw a ReferenceError when accessed before declaration.
- D is incorrect because JavaScript does not automatically assign `null`.

During execution, JavaScript first hoists the declaration `var a` and sets it to `undefined`. Only later does it assign the value 10.

Q2.

```
console.log(b);
let b = 20;
```

Options:
A. 20
B. undefined
C. ReferenceError
D. null

Answer: C

Explanation:

- A is incorrect because `b` is not initialized yet.
- B is incorrect because `let` does not initialize variables with `undefined` during hoisting.
- C is correct because of the Temporal Dead Zone (TDZ).
- D is incorrect as no value is assigned.

Variables declared with `let` are hoisted but not initialized. Accessing them before declaration results in a ReferenceError.

Q3.

```
console.log(typeof null);
```

Options:
A. null
B. undefined
C. object
D. number

Answer: C

Explanation:

- A is incorrect because `typeof` always returns a string.
- B is incorrect because `null` is not `undefined`.
- C is correct due to a historical bug in JavaScript.
- D is incorrect because `null` is not treated as a number.

Even though `null` is a primitive, JavaScript incorrectly reports its type as "object".

Q4.

```
console.log(5 == "5");
```

Options:
A. false
B. true
C. Error
D. undefined

Answer: B

Explanation:

- A is incorrect because loose equality allows coercion.
- B is correct because `"5"` is converted to number 5.
- C is incorrect because no error occurs.
- D is incorrect as a boolean result is returned.

The == operator performs implicit type conversion, making both values equal.

Q5.

```
console.log(5 === "5");
```

Options:
A. true
B. false
C. Error
D. undefined

Answer: B

Explanation:

- A is incorrect because types differ.
- B is correct since strict equality checks both type and value.
- C is incorrect as no error occurs.
- D is incorrect because a boolean is returned.

The number 5 and string "5" are not strictly equal due to type mismatch.

Q6.

```
function test() {
  console.log(this);
}
test();
```

Options:
A. undefined
B. window/global
C. null
D. function

Answer: B

Explanation:

- A is incorrect because this happens only in strict mode.
- B is correct because in non-strict mode, `this` refers to the global object.
- C is incorrect since `this` is never null by default.
- D is incorrect because `this` does not refer to the function itself.

The value of `this` depends on how the function is called.

Q7.

```
const obj = {
  name: "JS",
  show: () => console.log(this.name)
};
obj.show();
```

Options:
A. JS
B. undefined
C. Error
D. null

Answer: B

Explanation:

- A is incorrect because arrow functions do not bind their own `this`.
- B is correct since `this` refers to the global scope.
- C is incorrect because no runtime error occurs.
- D is incorrect because no null assignment exists.

Arrow functions inherit `this` from the surrounding lexical scope, not the object.

Q8.
```
let x = "5" - 2;
```

Options:
A. "3"
B. 3
C. NaN
D. Error

Answer: B

Explanation:

- A is incorrect because result is numeric.
- B is correct since subtraction forces numeric conversion.
- C is incorrect because conversion succeeds.
- D is incorrect as no error occurs.

JavaScript converts `"5"` to number 5, resulting in 3.

Q9.
```
console.log(NaN === NaN);
```

Options:
A. true
B. false

C. undefined
D. Error

Answer: B

Explanation:

- A is incorrect because NaN is never equal to itself.
- B is correct according to IEEE standard.
- C is incorrect since a boolean is returned.
- D is incorrect because no error occurs.

NaN is a special value and requires `Number.isNaN()` for checking.

Q10.

Spread operator

Options explained:

- A is correct because spread copies elements into a new array.
- B is incorrect as spread syntax is valid.
- C and D are incorrect since a valid array is returned.

Spread operator creates a shallow copy of arrays or objects.

Q11.

Rest parameter

Explanation:

- A is incorrect because all arguments are counted.
- B is correct since 4 arguments are passed.
- C and D are incorrect.

Rest parameters gather all arguments into an array.

Q12.

Destructuring

Explanation:

- A is incorrect since first element is assigned to `a`.
- B is correct.
- C and D are incorrect.

Destructuring assigns values based on position.

Q13.

IIFE

Explanation:

- A is incorrect because syntax is valid.
- B is correct as function runs immediately.
- C and D are incorrect.

IIFE executes instantly after definition.

Q14.

Closure

Explanation:

- A is incorrect since count increments.
- B is correct.
- C and D are incorrect.

Closures preserve variables from outer scope even after execution.

Q15.

typeof undefined

Explanation:

- A is correct.
- Others are incorrect types.

`typeof undefined` always returns "undefined".

Q16.

Falsy value

Explanation:

- A is incorrect because 0 is falsy.
- B is correct.
- Others are incorrect.

Falsy values evaluate to false in conditions.

Q17.

Object spread

Explanation:

- A and B are partial results.
- C is correct merged object.
- D is incorrect.

Spread merges properties into new object.

Q18.

Currying

Explanation:

- A is correct because function returns another function.
- Others are incorrect.

Currying transforms multi-argument function into chained calls.

Q19.

Array push

Explanation:

- A is incorrect because element is added.
- B is correct.
- Others are incorrect.

Push increases array length.

Q20.

```
[] == false
```

Explanation:

- A is correct due to type coercion.
- B is incorrect.
- Others are incorrect.

Empty array converts to empty string, then to 0, same as false.

✅ Part 2: Non-Coding MCQs (Detailed)

Q21.

- A incorrect because JS is interpreted.
- B incorrect because not statically typed.
- C correct since JS is dynamically typed.
- D incorrect.

Dynamic typing allows flexible variable types.

Q22.

- A, B, D are frontend frameworks.
- C is correct as Node.js runs JS on server.

Node.js enables backend development.

Q23.

- A incorrect because `var` is function-scoped.
- B correct.
- C incorrect.
- D incorrect.

`let` and `const` are block scoped.

Q24.

- A incorrect.
- B correct definition.
- C and D incorrect.

Closures help retain outer variables.

Q25.

- A, B, D are falsy.
- C is correct since arrays are truthy.

Truthy values evaluate to true in conditions.

Q26.

- A, B, D incorrect.
- C correct.

NaN represents invalid numeric operations.

Q27.

- A, C, D incorrect.
- B correct.

Spread expands iterable elements.

Q28.

- A incorrect due to side effects.
- B incorrect.
- C correct.
- D incorrect.

Pure functions are predictable and testable.

Q29.

- A incorrect.

- B correct.
- C, D incorrect.

Recursion solves problems via self-calls.

Q30.

- A, C, D incorrect.
- B correct.

Call stack manages execution order using LIFO.

Aptitude Question (percentage, average, ratio and proportion)

PART 1: Percentage (10 Questions)

Q1. What is 20% of 250?

A. 40
B. 50
C. 60
D. 70

Answer: B

Solution:
20% means 20/100. So, 20/100 × 250 = 50.
You can also think of 10% of 250 = 25, so 20% = 25 × 2 = 50.
Hence, the correct answer is 50.

Q2. A number increases from 100 to 120. What is the percentage increase?

A. 10%
B. 15%
C. 20%
D. 25%

Answer: C

Solution:
Increase = 120 − 100 = 20.
Percentage increase = (20/100) × 100 = 20%.
Always divide increase by original value.

Q3. Find 15% of 80.

A. 10
B. 12
C. 14
D. 16

Answer: B

Solution:
15% = 15/100.
So, 15/100 × 80 = 12.
Alternatively, 10% of 80 = 8 and 5% = 4 → total = 12.

Q4. A number is decreased by 25% and becomes 75. What was the original number?

A. 80
B. 90
C. 100
D. 120

Answer: C

Solution:
After 25% decrease, value becomes 75% of original.
So, 75% = 75 → original = 75 / 0.75 = 100.
Thus original number is 100.

Q5. What is 50% of 300?

A. 100
B. 120
C. 150
D. 200

Answer: C

Solution:
50% means half.
Half of 300 = 150.
So, directly answer is 150.

Q6. A student scored 45 out of 60. Find percentage.

A. 70%
B. 75%
C. 80%
D. 85%

Answer: B

Solution:
Percentage = (45/60) × 100.
= 75%.
Divide first → 45 ÷ 60 = 0.75.

Q7. What is 10% of 450?

A. 40
B. 45
C. 50
D. 55

Answer: B

Solution:
10% means divide by 10.
450 ÷ 10 = 45.
So answer is 45.

Q8. Price increased from 200 to 260. % increase?

A. 20%
B. 25%

C. 30%
D. 35%

Answer: C

Solution:
Increase = 260 − 200 = 60.
% increase = (60/200) × 100 = 30%.
Always compare with original.

Q9. 40% of a number is 200. Find number.

A. 400
B. 450
C. 500
D. 600

Answer: C

Solution:
40% = 200.
Number = 200 / 0.4 = 500.
So original number is 500.

Q10. What is 5% of 600?

A. 20
B. 25
C. 30
D. 35

Answer: C

Solution:
5% = 5/100.
600 × 5/100 = 30.
Also 10% is 60, so half = 30.

Q11. Average of 2, 4, 6, 8?

A. 4
B. 5
C. 6
D. 7

Answer: B

Solution:
Sum = 2+4+6+8 = 20.
Average = 20/4 = 5.
Average = total / number of values.

Q12. Average of 5 numbers is 20. Total sum?

A. 80
B. 90
C. 100
D. 110

Answer: C

Solution:
Average × number = total.
20 × 5 = 100.
So total sum is 100.

Q13. Average of 10 numbers is 15. Find sum.

A. 120
B. 130
C. 150
D. 160

Answer: C

Solution:
Sum = 15 × 10 = 150.
Direct formula: total = average × count.

Q14. Average of 3 numbers is 30. Find total.

A. 60
B. 70
C. 80
D. 90

Answer: D

Solution:
Total = 30 × 3 = 90.
Always multiply average by count.

Q15. Average of 20 and 40?

A. 20
B. 25
C. 30
D. 35

Answer: C

Solution:
(20 + 40)/2 = 60/2 = 30.
Average of two numbers is midpoint.

Q16. If average is 50 for 4 numbers, sum?

A. 150
B. 200
C. 250
D. 300

Answer: B

Solution:
Sum = 50 × 4 = 200.
Formula: sum = average × count.

Q17. Average of 6 numbers is 10. Add 1 number (20). New average?

A. 11
B. 12
C. 13
D. 14

Answer: B

Solution:
Old sum = 6 × 10 = 60.
New sum = 60 + 20 = 80.
New average = 80/7 ≈ 11.43 ≈ 12 (closest).

Q18. Average of 3, 6, 9?

A. 5
B. 6
C. 7
D. 8

Answer: B

Solution:
Sum = 18.
Average = 18/3 = 6.
Simple arithmetic mean.

Q19. Average of 4 numbers is 25. One number is 10. Remaining sum?

A. 70
B. 80
C. 90
D. 100

Answer: C

Solution:
Total sum = 25 × 4 = 100.
Remaining = 100 − 10 = 90.

Q20. Average of 1 to 9?

A. 4
B. 5
C. 6
D. 7

Answer: B

Solution:
For consecutive numbers: average = (first + last)/2.
(1 + 9)/2 = 5.
Shortcut formula.

Q21. Ratio of 10:20 simplifies to?

A. 1:2
B. 2:3
C. 3:4
D. 1:3

Answer: A

Solution:
Divide both by 10 → 10:20 = 1:2.
Simplify ratios like fractions.

Q22. If a:b = 2:3 and b:c = 4:5, find a:c.

A. 2:5
B. 4:5
C. 8:15
D. 10:15

Answer: C

Solution:
Make b same:
2:3 and 4:5 → multiply first by 4 → 8:12
Second by 3 → 12:15
So a:c = 8:15.

Q23. Divide 100 in ratio 2:3.

A. 40, 60
B. 50, 50
C. 30, 70
D. 20, 80

Answer: A

Solution:
Total parts = 5.
First = (2/5)×100 = 40
Second = (3/5)×100 = 60.

Q24. Ratio 3:4, if first is 30, second?

A. 35
B. 40
C. 45
D. 50

Answer: B

Solution:
3 → 30 means multiply by 10.
So 4 × 10 = 40.

Q25. If x:y = 5:2 and x=50, find y.

A. 10
B. 20
C. 25
D. 30

Answer: B

Solution:
5 → 50 means multiply by 10.
So 2 × 10 = 20.

Q26. Ratio of 15 and 45?

A. 1:2
B. 1:3
C. 2:3
D. 3:5

Answer: B

Solution:
Divide both by 15 → 1:3.

Q27. If a:b = 4:5, total = 90. Find a.

A. 30
B. 40
C. 50
D. 60

Answer: B

Solution:
Total parts = 9.
a = (4/9)×90 = 40.

Q28. If ratio is 7:3, difference = 40. Find numbers.

A. 70, 30
B. 60, 20
C. 80, 40
D. 90, 50

Answer: A

Solution:
Difference parts = 4.
4 parts = 40 → 1 part = 10.
Numbers = 70 and 30.

Q29. If a:b = 1:2, and b:c = 3:4, find a:c.

A. 1:4
B. 3:4
C. 3:8
D. 1:8

Answer: C

Solution:
Match b:
1:2 and 3:4 → multiply first by 3 → 3:6
Second by 2 → 6:8
So a:c = 3:8.

Q30. Divide 60 in ratio 1:2:3.

A. 10, 20, 30
B. 15, 20, 25
C. 5, 10, 15
D. 20, 20, 20

Answer: A

Solution:
Total parts = 6.
Values: (1/6)×60=10, (2/6)=20, (3/6)=30.
Hence result is 10, 20, 30.

Programming Question

IF-ELSE PROGRAMS (1–10)

1. Positive, Negative, or Zero

Logic:
Check if number > 0, < 0, or equal to 0.

Example:
Input: -5
Output: Negative number

---

2. Even or Odd

Logic:
Check if number % 2 == 0 → even, else odd

Example:
Input: 8
Output: Even

---

3. Largest Among Three Numbers

Logic:
Compare all three numbers using conditions.

Example:
Input: 5, 9, 3
Output: Largest is 9

---

4. Leap Year

Logic:

- Divisible by 4 AND not by 100
- OR divisible by 400

Example:
Input: 2024
Output: Leap Year

---

5. Vowel or Consonant

Logic:
Check if character is a, e, i, o, u (case-insensitive).

Example:
Input: e
Output: Vowel

---

6. Voting Eligibility

Logic:
If age ≥ 18 → eligible

Example:
Input: 17
Output: Not eligible

---

7. Divisible by 5 and 11

Logic:
Check number % 5 == 0 AND number % 11 == 0

Example:
Input: 55
Output: Divisible by both

---

8. Student Grade

Logic:
Assign grade based on marks range.

Example:
Input: 82
Output: Grade B

---

9. Three-Digit Number

Logic:
Check if number is between 100 and 999 (or negative equivalent).

Example:
Input: 456
Output: Three-digit number

---

10. Profit or Loss

Logic:
Compare selling price (SP) and cost price (CP).

Example:
Input: CP=100, SP=120
Output: Profit = 20

---

11. Print 1 to 100

Logic:
Loop from 1 to 100 and print each number.

Example Output:
1 2 3 ... 100

---

12. Print 100 to 1

Logic:
Loop from 100 down to 1.

Example Output:
100 99 98 ... 1

---

13. Even Numbers (1–50)

Logic:
Print numbers divisible by 2 in range.

Example Output:
2 4 6 ... 50

---

14. Odd Numbers (1–50)

Logic:
Print numbers not divisible by 2.

Example Output:
1 3 5 ... 49

---

15. Sum of First N Numbers

Logic:
Add numbers from 1 to N.

Example:
Input: 5
Output: 15

◈ Intermediate

16. Factorial

Logic:
Multiply numbers from 1 to N.

Example:
Input: 5
Output: 120

---

17. Multiplication Table

Logic:
Multiply number with 1–10.

Example:
Input: 3
Output: 3, 6, 9 ... 30

---

18. Count Digits

Logic:
Divide number by 10 repeatedly and count steps.

Example:
Input: 1234
Output: 4 digits

---

19. Reverse Number

Logic:
Extract digits and rebuild reversed number.

Example:
Input: 123
Output: 321

---

20. Palindrome

Logic:
Reverse number and compare with original.

Example:
Input: 121
Output: Palindrome

---

21. Sum of Digits

Logic:
Add each digit of number.

Example:
Input: 123
Output: 6

---

22. Prime Number

Logic:
Check divisibility from 2 to n-1 (or √n).

Example:
Input: 7
Output: Prime

---

23. Prime Numbers up to N

Logic:
Check each number from 1 to N.

Example:
Input: 10
Output: 2 3 5 7

---

24. Fibonacci Series

Logic:
Next number = sum of previous two.

Example:
Input: 5
Output: 0 1 1 2 3

---

25. Largest Digit

Logic:
Extract digits and compare.

Example:
Input: 5832
Output: 8

---

26. Right-Angled Triangle

Logic:
Nested loop → increase stars per row.

Example:

```
*
**
***
****
```

---

27. Inverted Triangle

Logic:
Decrease stars per row.

Example:

```
****
***
**
*
```

---

28. Power of Number

Logic:
Multiply base repeatedly exponent times.

Example:
Input: 2^3
Output: 8

---

29. GCD of Two Numbers

Logic:
Find largest number dividing both numbers.

Example:
Input: 12, 18
Output: 6

---

30. Armstrong Number

Logic:
Sum of digits raised to power of number of digits equals original number.

Example:
Input: 153
Output: Armstrong number
($1^3 + 5^3 + 3^3 = 153$)